

leanprover-community.github.io

Documentation style

10–12 minutes

All pull requests must meet the following documentation standards. See [the doc-gen repo](#) for information about the [automatically generated doc pages](#).

You can preview the markdown processing of a GitHub page or pull request using the [Lean doc preview page](#).

Each mathlib file should start with:

- a header comment with copyright information (see the [recommendations in our style guidelines](#));
- the list of imports (one on each line);
- a module docstring containing general documentation, written [using Markdown and LaTeX](#).

(See the example below.)

Headers use atx-style headers (with hash signs, no underlying dash). The open and close delimiters `/ - !` and `- /` should appear on their own lines.

The mandatory title of the file is a first level header. It is followed by a summary of the content of the file.

The other sections, with second level headers are (in this order):

- *Main definitions* (optional, can be covered in the summary)
- *Main statements* (optional, can be covered in the summary)
- *Notation* (omitted only if no notation is introduced in this file)
- *Implementation notes* (description of important design decisions or interface features, including use of type classes and simp canonical form for new definitions)
- *References* (references to textbooks or papers, or Wikipedia pages)
- *Tags* (a list of keywords that could be useful when doing text search in mathlib to find where something is covered)

References should refer to bibtex entries in [the mathlib citations file](#). See the [Citing other works](#) section below.

The following code block is an example of a file header.

```
/-
Copyright (c) 2018 Robert Y. Lewis. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Robert Y. Lewis
-/
module

public import Mathlib.Algebra.Order.AbsoluteValue.Basic
```

```

public import Mathlib.NumberTheory.Padics.PadicVal.Basic

/-!
# p-adic norm

This file defines the `p`-adic norm on `ℚ`.

The `p`-adic valuation on `ℚ` is the difference of the multiplicities of `p` in
the numerator and
denominator of `q`. This function obeys the standard properties of a valuation,
with the appropriate
assumptions on `p`.

The valuation induces a norm on `ℚ`. This norm is a nonarchimedean absolute
value.
It takes values in  $\{0\} \cup \{1/p^k \mid k \in \mathbb{Z}\}$ .

## Implementation notes

Much, but not all, of this file assumes that `p` is prime. This assumption is
inferred automatically
by taking `[Fact p.Prime]` as a type class argument.

## References

* [F. Q. Gouvêa, *p-adic numbers*][gouvea1997]
* [R. Y. Lewis, *A formal proof of Hensel's lemma over the p-adic integers*]
[lewis2019]
* <https://en.wikipedia.org/wiki/P-adic_number>

## Tags

p-adic, p adic, padic, norm, valuation
-/

```

Doc strings <#>

Every definition and major theorem is required to have a doc string. (Doc strings on lemmas are also encouraged, particularly if the lemma has any mathematical content or might be useful in another file.) These are introduced using `/--` and closed by `-/` above the definition, with either newlines or single spaces between the markers and the text. Subsequent lines in a doc-string should not be indented. They can contain Markdown and LaTeX as well: see the next section. If a doc string is a complete sentence, then it should end in a period. Named theorems, such as the **mean value theorem** should be bold faced (i.e., with two asterisks before and after).

Doc strings should convey the mathematical meaning of the definition. They are allowed to lie slightly about the actual implementation. The following is a doc string example:

```

/-- If `q ≠ 0`, the `p`-adic norm of a rational `q` is `p ^ (-padicValRat p
q)` .
If `q = 0`, the `p`-adic norm of `q` is `0`. -/
def padicNorm (p : ℕ) (q : ℚ) : ℚ :=
  if q = 0 then 0 else (p : ℚ) ^ (-padicValRat p q)

```

An example that is slightly lying but still describes the mathematical content would be:

```
/-- `padicValRat` defines the valuation of a rational `q` to be the valuation
of `q.num` minus the
valuation of `q.den`. If `q = 0` or `p = 1`, then `padicValRat p q` defaults to
`0`. -/
def padicValRat (p : ℕ) (q : ℚ) : ℤ :=
  padicValInt p q.num - padicValNat p q.den
```

Tactic documentation <#>

Tactics should have a docstring that matches the [Lean documentation style guide](#). Key points are: Be complete and self-contained but brief. The docstring should start with a full sentence that has the tactic as the subject. (Example: "rewrite [e] uses the expression e as a rewrite rule on the main goal.") All different options and forms of the tactic should appear in a bulleted list. The "Examples:" section, if any, should be a sequence of code blocks.

Linting <#>

The docBlame linter lists all definitions that do not have doc strings. The docBlameThm linter will lists theorems and lemmas that do not have doc strings. The tacticDocs linter lists all tactics that do not have docstrings.

To run only the docBlame linter, add the following to the end of your lean file:

```
#lint only docBlame
```

To run only the docBlame and docBlameThm linters, add the following to the end of your lean file:

```
#lint only docBlame docBlameThm
```

To run the all default linters, including docBlame and tacticDocs, add the following to the end of your lean file:

```
#lint
```

To run the all default linters, including docBlame and run docBlameThm, add the following to the end of your lean file:

```
#lint docBlameThm
```

LaTeX and Markdown <#>

We generally put references to Lean declarations or variables in between backticks. Writing the fully-qualified name (e.g. `finset.card_pos` instead of just `card_pos`) will turn the name into a link on our [online docs](#).

Raw URLs should be enclosed in angle brackets `<...>` to ensure that they will be clickable online. (Some URLs, especially those with parentheses or other special symbols, may not be parsed correctly by the markdown renderer.)

When talking about mathematical symbols instead, it may be preferable to use LaTeX. LaTeX can be included in doc strings in three ways:

- using single dollar signs `$ ... $` to render math inline,
- using double dollar signs `$$ ... $$` to render math in "display mode", or
- using environments `\begin{...} ... \end{...}` (without dollar signs).

These correspond to the [MathJax](#) settings of our online docs. The interaction between the Markdown and LaTeX there is similar to that on <https://math.stackexchange.com> and <https://mathoverflow.net>, so you can paste a doc string into [an editing sandbox there](#) to preview the final result. See also the

math.stackexchange [Mathjax tutorial](#).

It is common to structure a file in sections, where each section contains related declarations. By describing the sections with module documentation `/-! ... -/` at the beginning, these sections can be seen in the documentation.

While these sectioning comments will often correspond to section or namespace commands, this is not required. You can use sectioning comments inside of a section or namespace, and you can have multiple sections or namespaces following one sectioning comment.

Sectioning comments are for display and readability only. They have no semantic meaning.

Third-level headers `###` should be used for titles inside sectioning comments.

If the comment is more than one line long, the delimiters `/-!` and `-/` should appear on their own lines.

See [Lean/Expr/Basic.lean](#) for an example in practice.

```
namespace BinderInfo

/-! ### Declarations about `BinderInfo` -/

/-- The brackets corresponding to a given `BinderInfo`. -/
def brackets : BinderInfo → String × String
| BinderInfo.implicit => ("{" , "}")
| BinderInfo.strictImplicit => ("{{" , "}}")
| BinderInfo.instImplicit => ("[" , "]" )
| _ => ("(" , ")")

end BinderInfo

namespace Name

/-! ### Declarations about `name` -/

/-- Find the largest prefix `n` of a `Name` such that `f n != none`, then
replace this prefix
with the value of `f n`. -/
def mapPrefix (f : Name → Option Name) (n : Name) : Name := Id.run do
  if let some n' := f n then return n'
  match n with
  | anonymous => anonymous
  | str n' s => mkStr (mapPrefix f n') s
  | num n' i => mkNum (mapPrefix f n') i
```

Theories documentation <#>

In addition to documentation living in Lean files, we have [theories documentation](#) where we give overviews spanning several Lean files, and more mathematical explanations in cases where formalization requires slightly exotic points of view, see for instance the [topology documentation](#).

Citing other works <#>

To cite papers and books in doc strings, the references should first be added to the BibTeX file: `docs/` `references.bib`. To normalize the file with `bibtool`, you can run:

```
bibtool --preserve.key.case=on --preserve.keys=on --print.use.tab=off --
```

```
pass.comments=on -s -i docs/references.bib -o docs/references.bib
```

To ensure that your citations become links in the online docs, you can use either of the following two styles:

First, you may enclose the citation key used in `docs/references.bib` in square brackets:

```
The proof can be found in [Boole1854].
```

In the online docs, this will become something like:

```
(The proof can be found in [Boo54]
```

(The key will change into an [alpha style label](#) and become a link to the [References page](#) of the docs.)

Alternatively, you can use custom text for the citation by putting text in square brackets ahead of the citation key:

```
See [Grundlagen der Geometrie][hilbert1999] for an alternative axiomatization.
```

```
(See Grundlagen der Geometrie for an alternative axiomatization.
```

Note that you currently cannot use the closing square bracket `]` symbol in the link text. So the following will not result in a working link:

```
We follow [Euclid's *Elements* [Prop. 1]][heath1956a].
```

```
(We follow [Euclid's Elements [Prop. 1]][heath1956a].
```

Language <#>

Documentation should be written in English. Any common spelling (e.g. British, American or Australian English) is acceptable. Pull requests should not be made that only change which of these spellings are used, but it is acceptable to change the spelling as part of a PR that substantially enhances the documentation. Contrast this with the rule for [declaration names](#), which should use American English spelling.

Examples <#>

The following files are maintained as examples of good documentation style:

- [Mathlib.NumberTheory.Padics.PadicNorm](#)
- [Mathlib.Topology.Basic](#)
- [Analysis.Calculus.ContDiff.Basic](#)